MLSM2154 – Gesture Recognition

0. Architecture générale du code

Le script est organisé en 13 sections numérotées, chacune correspondant à une étape du pipeline de machine learning :

- Section 1 — Chargement des données (load_domain1, load_domain4)
- Section 2 — Visualisation exploratoire (plot_sequence_lengths, plot_gesture_samples)
- Section 3 — Pré-traitement (standardize_gestures, apply_pca, cluster_and_encode)
- Section 4 — Méthodes baselines (dtw_distance, edit_distance)
- Section 5 — Classifieur k-NN (knn_predict)
- Section 6 — Extraction de features (extract_features, build_feature_dataset)
- Section 7 — Random Forest (random_forest_evaluation, random_forest_evaluation_user_dependent)
- Section 8 — LSTM (prepare_lstm_data, _build_lstm_model, lstm_evaluation, lstm_evaluation_user_dependent)
- Section 9 — Validation croisée baselines (crossval_user_independent, crossval_user_dependent)
- Section 10 — Matrices de confusion (compute_confusion_matrix, _rf, _lstm, draw_best_model_cm)
- Section 11 — Sauvegarde des résultats (save_fold_results)
- Section 12 — Tests statistiques (wilcoxon_test, generate_pvalue_table)
- Section 13 — Main (pipeline complet)

---

1. Chargement des données

1.1 load_domain1(folder_path)

Rôle : Lire tous les fichiers CSV du Domain 1 (chiffres 0–9 dessinés en 3D).

Chaque fichier suit la convention de nommage :

```
SubjectS-G-R.csv   où  S = sujet (1-10),  G = geste (0-9),  R = répétition (1-10)
```

La fonction :

- **Parse le nom de fichier** avec une expression régulière (regex) pour extraire S, G et R.
- Lit le CSV avec pandas en nommant les colonnes `x, y, z, t`.
- Ignore la colonne temporelle `t` (les consignes demandent de considérer des pas de temps constants).
- Convertit l'indice de sujet de 1-based à 0-based (`subject = S - 1`).
- Retourne trois listes parallèles : data (tableaux NumPy T×3), labels (entiers), users (entiers).

1.2 load_domain4(folder_path)

Rôle : Lire tous les fichiers texte du Domain 4 (figures 3D : cube, cylindre, sphère…).

Ces fichiers n'ont pas d'extension et contiennent un en-tête structuré :

```
Domain id = 4
Class id  = C
User id   = U
<x>,<y>,<z>,<t>
...données...
```

La fonction parse l'en-tête ligne par ligne, puis extrait les coordonnées 3D (x, y, z) en ignorant t.

1.3 check_completeness(labels, users, domain_name)

Rôle : Vérifier que chaque paire (utilisateur, geste) dispose exactement de 10 répétitions.

Utilise un dictionnaire de comptage (defaultdict). Affiche un avertissement pour toute paire incomplète. Cela garantit l'intégrité des données avant la validation croisée.

1.4 print_dataset_info(data, labels, users, domain_name)

Rôle : Afficher un résumé statistique : nombre de séquences, sujets, classes de geste, longueurs min/max/moyenne/std.

---

2. Visualisation exploratoire

2.1 plot_sequence_lengths(data, labels, domain_name, save_path)

Rôle : Afficher un box-plot des longueurs de séquences par classe de geste.

Chaque boîte montre la médiane, les quartiles Q1/Q3 et les valeurs aberrantes. Cela permet d'identifier si les séquences ont des longueurs très variables (ce qui justifie l'utilisation du DTW ou du padding pour le LSTM).

2.2 plot_gesture_samples(data, labels, users, domain_name, n_classes, n_subjects, save_path)

Rôle : Tracer les trajectoires 3D de plusieurs répétitions d'un même geste pour plusieurs sujets.

Chaque sous-graphe est une grille (classe × sujet). Les 3 premières répétitions sont superposées. Le point de départ est en vert, le point d'arrivée en rouge. Cela permet de visualiser la variabilité intra-classe et inter-sujets.

---

3. Pré-traitement

3.1 Standardisation — standardize_gestures(data)

Concept mathématique

La standardisation (ou normalisation z-score) transforme chaque dimension d'un geste pour qu'elle ait une moyenne nulle et un écart-type de 1 :

$$x'_i = (x_i - \mu) / \sigma$$

où μ est la moyenne et σ l'écart-type calculés sur tous les points temporels du geste.

Explication pédagogique

Imaginez que deux utilisateurs tracent le chiffre '3' : l'un devant soi à hauteur de taille, l'autre au-dessus de sa tête. Les coordonnées absolues (x, y, z) seront très différentes, mais la forme du geste est identique. La standardisation supprime ces différences de position et d'échelle, ne conservant que la forme.

Dans le code, la standardisation est faite par geste indépendamment (pas globalement) pour éviter que les données de test ne contaminent les paramètres calculés sur le train.

3.2 Analyse en Composantes Principales — apply_pca(data, n_components=2)

Concept mathématique

La PCA est une transformation linéaire orthogonale. Soit X une matrice N×3 de tous les points 3D de tous les gestes. La PCA cherche les directions (composantes principales) qui maximisent la variance des données :

$$X = U \Sigma V^T \quad \text{(décomposition en valeurs singulières)}$$

Les n_components premières colonnes de V forment la matrice de projection W (3×n). Chaque point est projeté :

$$x_projeté = x \cdot W$$

Explication pédagogique

La PCA est comme trouver le meilleur angle de vue d'un objet 3D. Si l'objet est un geste, on cherche la direction depuis laquelle on voit le plus de variabilité. En gardant 2 composantes, on passe de 3 dimensions à 2, ce qui réduit le bruit et accélère les calculs du k-means. La variance expliquée indique combien d'information est conservée.

Dans notre code, la PCA est ajustée sur l'ensemble complet du dataset (autorisé explicitement par les consignes) et le résultat de la variance expliquée est affiché pour chaque domaine.

3.3 Encodage symbolique — cluster_and_encode(data_pca, k=20)

Concept mathématique

Le k-means partitionne N points en k clusters en minimisant la somme des distances au carré entre chaque point et son centroïde :

$$\text{argmin_}C \ \Sigma_i \Sigma_{j \in C_i} ||x_j - \mu_i||^2$$

Chaque point est ensuite remplacé par l'étiquette de son centroïde le plus proche. Un geste de T points devient une séquence de T entiers dans {0, …, k-1}, par exemple : 0 0 3 3 3 7 7 2 2 0…

Explication pédagogique

C'est comme créer un alphabet de k symboles qui représentent des 'positions types' dans l'espace réduit. Chaque geste devient alors un mot dans cet alphabet, ce qui permet d'utiliser la distance d'édition (conçue pour comparer des chaînes de symboles).

---

4. Méthodes baselines

4.1 Dynamic Time Warping — dtw_distance(seq1, seq2)

Concept mathématique

Le DTW est un algorithme de programmation dynamique qui mesure la similarité entre deux séquences temporelles en autorisant des alignements non linéaires. Soit seq1 de longueur I et seq2 de longueur J. On construit une matrice d'accumulation g(i, j) :

$$g(0, 0) = 0, \quad g(i, 0) = g(0, j) = +\infty$$

$$g(i, j) = d(seq1_i, seq2_j) + \min\{ g(i\text{-}1, j),\ 2 \cdot d + g(i\text{-}1, j\text{-}1),\ g(i, j\text{-}1) \}$$

où $d(a, b) = ||a - b||_2$ est la distance euclidienne entre deux points 3D. Le facteur ×2 sur la diagonale suit la formulation du cours. La distance finale est normalisée :

$$D(seq1, seq2) = g(I, J) / (I + J)$$

Explication pédagogique

Imaginez deux personnes qui tracent le chiffre '1' : l'une lentement, l'autre rapidement. Leurs séquences temporelles ont des longueurs différentes et un simple alignement point-à-point échouerait. Le DTW 'étire' ou 'compresse' les séquences pour trouver le meilleur alignement possible, comme un accordéon. C'est pourquoi il est très adapté aux gestes où la vitesse d'exécution varie.

Dans le code, la fonction est compilée avec @njit (Numba) pour une exécution très rapide en C compilé, car le DTW a une complexité O(I×J) et doit être calculé pour chaque paire (test, train).

4.2 Distance d'édition — edit_distance(seq1, seq2)

Concept mathématique

La distance d'édition (Levenshtein) entre deux séquences de symboles compte le nombre minimal d'opérations (insertion, suppression, substitution) pour transformer seq1 en seq2 :

$$ED(i, 0) = i, \quad ED(0, j) = j$$

$$ED(i, j) = \min\{ ED(i\text{-}1, j) + 1,\ ED(i, j\text{-}1) + 1,\ ED(i\text{-}1, j\text{-}1) + \delta(s_{1i}, s_{2j}) \}$$

où $\delta(a, b) = 0$ si a = b, 1 sinon.

Explication pédagogique

La distance d'édition est l'algorithme derrière la correction orthographique : combien de 'fautes de frappe' séparent deux mots ? Ici les 'mots' sont des séquences de symboles k-means. Deux gestes similaires parcourront des clusters similaires dans le même ordre, donc leur distance d'édition sera faible.

Contrairement au DTW qui travaille sur des coordonnées continues, l'Edit Distance nécessite d'abord la vectorisation quantifiée (k-means) pour produire des séquences symboliques comparables.

5. Classifieur k-NN — knn_predict(test_item, train_items, train_labels, distance_fn, k)

Concept mathématique

Le k-NN (k plus proches voisins) est un classifieur non paramétrique. Pour classer un élément de test x :

- Calculer $d(x, x_i)$ pour tous les x_i dans le jeu d'entraînement.
- Sélectionner les k indices avec les distances les plus petites : $N_k(x) = argsort(distances)[:k]$
- Retourner la classe majoritaire parmi ces k voisins : $\hat{y} = argmax_c\ |\{i \in N_k(x) : y_i = c\}|$

Explication pédagogique

Le k-NN est le classifieur le plus intuitif : 'dis-moi qui sont tes voisins, je te dirai qui tu es.' Pour un geste inconnu, on cherche les gestes d'entraînement les plus similaires (selon DTW ou Edit Distance) et on vote pour la classe la plus représentée parmi eux. Avec k=1, on prend simplement le geste le plus proche.

Dans le code, la parallélisation (joblib.Parallel) permet de calculer les distances pour tous les éléments de test simultanément sur tous les cœurs du processeur.

6. Extraction de features — extract_features(sequence)

Cette fonction transforme une trajectoire 3D de longueur variable en un vecteur fixe de 42 features numériques, utilisable par des classifieurs classiques comme le Random Forest.

6.1 Statistiques par axe (21 features : 7 × 3 axes)

Pour chacun des axes x, y, z on calcule :

- **Moyenne** $\mu = (1/T)\ \Sigma\ x_i$ — position centrale du geste sur cet axe.
- **Écart-type** $\sigma = \sqrt{(1/T)\ \Sigma\ (x_i - \mu)^2}$ — amplitude des mouvements.
- **Min et Max** — bornes de l'amplitude.
- **Range** = $max - min$ — étendue totale.
- **Skewness (asymétrie)** = $E[(X-\mu)^3] / \sigma^3$ — la distribution est-elle symétrique ou penchée d'un côté ?
- **Kurtosis (aplatissement)** = $E[(X-\mu)^4] / \sigma^4 - 3$ — la distribution est-elle pointue ou aplatie ?

6.2 Vitesse et accélération (6 features)

La vitesse instantanée est calculée comme la différence première : $v_i = ||p_{i+1} - p_i||_2$. On en extrait mean, std, max, min. L'accélération est la différence seconde : $a_i = ||v_{i+1} - v_i||_2$. On en extrait mean et std.

6.3 Géométrie globale (5 features)

- **Longueur du chemin** = $\Sigma\ ||v_i||$ — distance totale parcourue.
- **Bounding box** — taille de la boîte englobante (3 dimensions + diagonale).

6.4 Énergie spectrale via FFT (9 features : 3 × 3 axes)

La Transformée de Fourier Rapide (FFT) décompose le signal en fréquences :

$$X[k] = \Sigma_n x[n] \cdot e^{(-2\pi i \cdot kn/N)}$$

On calcule |X[k]| (le module) et on en extrait mean, std, max par axe. Cela capture les patterns rythmiques du geste (ex. une courbe rapide vs. lente).

---

7. Random Forest

7.1 Concept mathématique

Le Random Forest est un ensemble (ensemble method) de B arbres de décision entraînés chacun sur un sous-échantillon bootstrap des données et un sous-ensemble aléatoire des features. La prédiction finale est le vote majoritaire :

$$\hat{y} = mode\{ T_1(x), T_2(x), ..., T_B(x) \}$$

Chaque arbre T_1 est construit en sélectionnant à chaque nœud le meilleur split parmi m features tirées aléatoirement ($m \approx \sqrt{p}$ pour la classification). La variance est réduite par la diversité des arbres (bagging + randomisation des features).

7.2 Explication pédagogique

Un seul arbre de décision est comme un questionnaire : 'la vitesse max est-elle > 2.3 ? si oui, le range sur z est-il > 1.1 ?…'. Il apprend bien les données d'entraînement mais généralise mal (overfitting). Le Random Forest construit 100 arbres différents (n_estimators=100), chacun avec des données et des features légèrement différentes, et vote. C'est la sagesse des foules : chaque arbre fait des erreurs différentes, et en votant ensemble les erreurs s'annulent.

7.3 Fonctions dans le code

random_forest_evaluation(data, labels, users)

Setting : User-independent (leave-one-user-out).

Pour chaque utilisateur u (10 au total) : on entraîne le RF sur les 9 autres utilisateurs et on teste sur u. On retourne mean et std des 10 accuracies.

random_forest_evaluation_user_dependent(data, labels, users)

Setting : User-dependent (leave-one-sample-out par utilisateur).

Pour chaque utilisateur u et chaque fold f (0 à 9) : le test set contient la f-ième répétition de chaque classe de cet utilisateur ; le train set contient les 9 autres répétitions du même utilisateur. Les 100 accuracies (10 users × 10 folds) sont retournées.

---

8. LSTM (Long Short-Term Memory)

8.1 Concept mathématique

Le LSTM est un type de réseau de neurones récurrent (RNN) conçu pour apprendre des dépendances à long terme dans des séquences. À chaque pas de temps t, le LSTM maintient un état caché h_t et une cellule mémoire c_t grâce à trois portes :

$$f_t = \sigma(Wf \cdot [h_{t-1}, x_t] + bf) \quad \leftarrow \textit{Porte d'oubli (forget gate)}$$
$$i_t = \sigma(Wi \cdot [h_{t-1}, x_t] + bi) \quad \leftarrow \textit{Porte d'entrée (input gate)}$$
$$o_t = \sigma(Wo \cdot [h_{t-1}, x_t] + bo) \quad \leftarrow \textit{Porte de sortie (output gate)}$$
$$\tilde{c}_t = tanh(Wc \cdot [h_{t-1}, x_t] + bc) \quad \leftarrow \textit{Candidat mémoire}$$
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad \leftarrow \textit{Mise à jour mémoire}$$
$$h_t = o_t \odot tanh(c_t) \quad \leftarrow \textit{État caché}$$

où σ est la fonction sigmoïde et $\odot$ le produit élément par élément.

8.2 Explication pédagogique

Un RNN classique 'oublie' les informations lointaines (problème du gradient qui s'évanouit). Le LSTM résout cela grâce à sa mémoire explicite (cellule c). Les trois portes contrôlent quoi oublier (f_t), quoi mémoriser (i_t) et quoi exposer en sortie (o_t). C'est comme un carnet de notes : on choisit quoi écrire, quoi effacer, et quoi lire.

Pour les gestes, le LSTM lit la trajectoire 3D point par point (x, y, z à chaque t) et apprend à reconnaître les patterns temporels qui distinguent un '3' d'un '8'.

8.3 Architecture dans le code (_build_lstm_model)

```
Input(shape=(T, 3))          ← T pas de temps, 3 coordonnées
Masking(mask_value=0.0)      ← Ignore les zéros de padding
LSTM(64)                     ← 64 unités cachées
Dense(32, relu)              ← Couche fully-connected
Dense(n_classes, softmax)    ← Probabilités par classe
```

Le padding est nécessaire car les séquences ont des longueurs variables. `pad_sequences` ajoute des zéros en fin de séquence pour uniformiser à max_len=150 pas de temps. Le Masking layer signale au LSTM d'ignorer ces zéros artificiels.

8.4 Entraînement

Le modèle est compilé avec l'optimiseur Adam (adaptatif), la loss sparse_categorical_crossentropy (pour les classes entières), et entraîné 30 epochs avec batch_size=32 et validation_split=0.1. Ces hyperparamètres sont des choix pratiques équilibrant vitesse et précision.

8.5 Fonctions dans le code

lstm_evaluation(data, labels, users)

Leave-one-user-out (user-independent). Pour chaque utilisateur u : un nouveau modèle LSTM est instancié, entraîné sur les 9 autres, puis évalué sur u. Cela donne 10 accuracies.

lstm_evaluation_user_dependent(data, labels, users)

Leave-one-sample-out (user-dependent). Pour chaque utilisateur u et fold f : le modèle entraîne uniquement sur les répétitions de cet utilisateur (les 9 autres) et teste sur la f-ième répétition. Cela donne 100 accuracies.

---

9. Validation croisée

9.1 crossval_user_independent(items, labels, users, distance_fn, k)

Principe — Leave-One-User-Out (LOUO)

Il y a 10 utilisateurs. À chaque fold f ∈ {0, …, 9} :

- **Test set :** toutes les séquences de l'utilisateur f (100 séquences : 10 gestes × 10 répétitions).
- **Train set :** toutes les séquences des 9 autres utilisateurs (900 séquences).

Le modèle est évalué sur un utilisateur qu'il n'a jamais vu. Cela mesure la généralisation à de nouveaux utilisateurs. On obtient 10 accuracies dont on calcule mean et std.

9.2 crossval_user_dependent(items, labels, users, distance_fn, k)

Principe — Leave-One-Sample-Out (LOSO) par utilisateur

Pour chaque utilisateur u et chaque fold f ∈ {0, …, 9} :

- **Test set :** la f-ième répétition de chacune des 10 classes de l'utilisateur u (10 séquences).
- **Train set :** les 9 autres répétitions de chaque classe de l'utilisateur u (90 séquences).

Le modèle est personnalisé par utilisateur : il a vu des exemples de cet utilisateur en entraînement. On s'attend donc à de meilleures performances qu'en user-independent. On obtient 100 accuracies (10 × 10).

---

10. Matrices de confusion

10.1 Concept

La matrice de confusion CM[i][j] indique combien de fois la classe i a été prédite comme classe j. La diagonale contient les bonnes prédictions. Les valeurs hors diagonale révèlent les confusions systématiques entre classes (ex. le '4' souvent confondu avec le '9').

10.2 Fonctions dans le code

compute_confusion_matrix(items, labels, users, distance_fn, k, title)

Réexécute le leave-one-user-out complet et collecte toutes les prédictions. Utilisée pour Edit Distance et DTW.

compute_confusion_matrix_rf(data, labels, users, title)

Même principe mais entraîne un Random Forest à chaque fold. Réentraîner le modèle assure que la matrice est cohérente avec l'évaluation principale.

compute_confusion_matrix_lstm(data, labels, users, title)

Raison de l'existence de cette fonction : Le LSTM nécessite une gestion spécifique des labels (décalage 0-indexed via labels_zi) et du padding. Sans cette fonction dédiée, si le LSTM est le meilleur modèle, il était impossible de générer sa matrice de confusion correctement. Les autres méthodes avaient déjà leurs fonctions.

La fonction reconvertit les prédictions 0-indexed vers l'espace de labels original pour l'affichage.

draw_best_model_cm(best_name, data_std, sequences, labels, users, domain)

Rôle central : Identifie dynamiquement le meilleur modèle parmi les 4 méthodes et appelle la bonne fonction de matrice de confusion.

---

11. Tests statistiques

11.1 Test de Wilcoxon — wilcoxon_test(folds_a, folds_b, name_a, name_b, domain)

Concept mathématique

Le test de Wilcoxon signé-rang est un test non-paramétrique qui compare deux distributions appariées. Soit d_i = folds_a[i] − folds_b[i] les différences par fold. On classe les $|d_i|$ par ordre croissant et on attribue des rangs. La statistique W est :

$$W = \Sigma_i \, sgn(d_i) \cdot R_i$$

où R_i est le rang de $|d_i|$. Sous H_0 (les deux méthodes sont équivalentes), W suit une distribution connue. La p-value mesure la probabilité d'observer W aussi extrême si H_0 est vraie.

Explication pédagogique

On ne peut pas utiliser un test t de Student car les 10 folds de cross-validation ne sont pas indépendants (les données se chevauchent). Le test de Wilcoxon est plus adapté : il compare les différences fold-par-fold sans supposer une distribution normale.

Si $p < 0.05$: les deux méthodes sont significativement différentes (on rejette H_0). La méthode avec la mean accuracy la plus haute est déclarée meilleure.

11.2 Correction de Bonferroni — generate_pvalue_table(methods_results, domain)

Problème des comparaisons multiples

Si on effectue k tests statistiques indépendants avec α = 0.05 chacun, la probabilité de se tromper au moins une fois (Type I error) est :

$$P(au\ moins\ une\ erreur) = 1 - (1 - \alpha)\text{^}k$$

Avec 4 méthodes et C(4,2) = 6 comparaisons possibles : $P \approx 1 - 0.95^6 \approx 26\%$ de faux positifs !

Correction

La correction de Bonferroni divise le seuil α par le nombre de comparaisons :

$$\alpha_corrigé = \alpha / k = 0.05 / 6 \approx 0.0083$$

On ne déclare une différence significative que si p < α_corrigé. Cela garantit que le taux d'erreur global reste contrôlé à 5%.

Dans le code

La fonction génère une matrice N×N de p-values (N = nombre de méthodes), n'incluant que les paires avec le même nombre de folds (sinon le test est invalide). Elle affiche le seuil de Bonferroni et sauvegarde la matrice en CSV.

---

12. Résultats & Sauvegarde

12.1 save_fold_results(fold_accs, method, setting, domain)

Rôle : Sauvegarder les accuracies par fold dans un fichier CSV (ex. results_domain1_user_independent_dtw.csv). Cela permet de réappliquer les tests statistiques ultérieurement sans réexécuter toute la validation croisée.

12.2 Tableau récapitulatif final (Section 17 du main)

Le main construit un DataFrame pandas avec toutes les combinaisons (Domaine × Setting × Méthode) et l'affiche en tableau pivot. Il est sauvegardé dans summary_results.csv. Ce tableau est directement utilisable pour le rapport.

---

13. Pipeline principal (main)

Le main exécute le pipeline complet dans l'ordre suivant, pour les deux domaines :

- Étape 0 — Warm-up Numba (compilation JIT des fonctions DTW et Edit Distance, ~5-10s la première fois)
- Étape 1 — Chargement des données Domain 1 et Domain 4
- Étape 2 — Visualisation exploratoire (longueurs, trajectoires 3D) pour les DEUX domaines
- Étape 3 — Pré-traitement Domain 1 : standardisation → PCA → k-means (k=20)
- Étapes 4–5 — Baselines + méthodes avancées en USER-INDEPENDENT (Domain 1)
- Étapes 6–7 — Baselines + méthodes avancées en USER-DEPENDENT (Domain 1)
- Étape 8 — Tests de Wilcoxon + tableau Bonferroni (Domain 1)
- Étape 9 — Matrice de confusion du meilleur modèle (Domain 1)
- Étapes 10–16 — Même pipeline complet pour Domain 4
- Étape 17 — Tableau récapitulatif final et sauvegarde CSV